

Fase 2: LLM Integration

SQS + Async Pattern + Mock vs Real LLM

Tutorial paso a paso

Proyecto Sentinel AcademIA
lFunknown

20 de junio de 2026

¿Qué construimos en este tutorial?

En la **Fase 1** teníamos un CRUD básico (POST/GET/LIST de quejas). En esta fase agregamos **análisis con IA** de forma asíncrona:

- POST /api/quejas persiste la queja y **encola** un mensaje
- Una Lambda **consumer de SQS** procesa la cola en background
- Esa Lambda llama al LLM (Cohere) y actualiza el status a ANALIZADA
- GET /api/quejas/{id} devuelve la queja con su análisis

Decisiones clave:

- **Por qué SQS:** desacopla el POST rápido (202) del análisis lento (LLM)
- **Por qué Mock LLM en Lambda:** el package oci pesa 229MB y excede el límite de 250MB de Lambda
- **Por qué Real LLM local:** para demos completas con Cohere real

Trampa de la fase: el límite de 250MB de Lambda

Cuando agregamos `pip install oci` a `requirements.txt`, el zip subió a **292MB** y Lambda rechazó el deploy con:

Unzipped size must be smaller than 262144000 bytes.

Solución: MockLLMClient (rule-based, sin deps externas) en Lambda, RealLLMClient solo local.

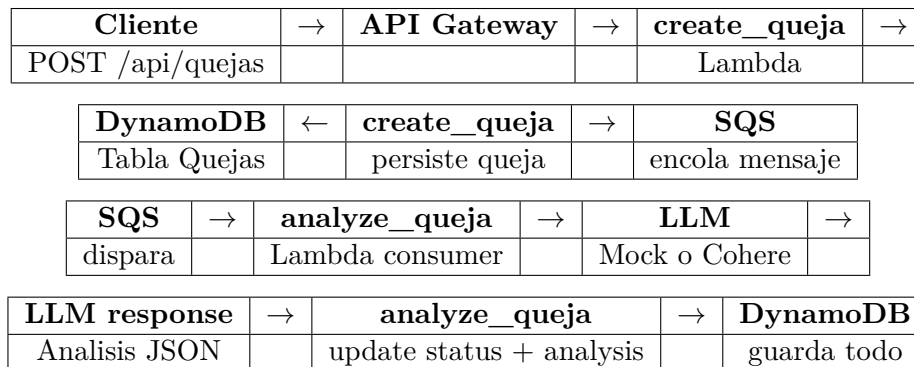
Índice

1. Arquitectura del flujo asíncrono	3
1.1. Diagrama del flujo	3
1.2. Status workflow (state machine)	3
2. Componente 1: SQS Queue + Dead Letter Queue	3
2.1. Qué es SQS	3
2.2. Configuración en SAM (template.yaml)	3
2.3. Event mapping (SQS → Lambda)	4
3. Componente 2: Analisis schema (Pydantic)	4
3.1. El schema alineado con OpenAPI	4

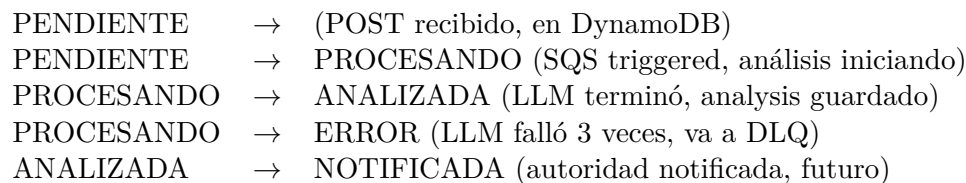
3.2. El mensaje SQS	5
4. Componente 3: create_queja ahora envía a SQS	5
4.1. El nuevo flujo	5
4.2. Integración en el handler	6
5. Componente 4: analyze_queja Lambda (SQS consumer)	6
5.1. El handler completo	6
5.2. Patrones importantes	8
6. Componente 5: LLM Client (Mock + Real)	8
6.1. El problema del 250MB	8
6.2. La solución: factory con toggle	8
7. MockLLMClient: rule-based	9
7.1. Algoritmo de clasificación	9
8. RealLLMClient: OCI Cohere (solo local)	10
8.1. Cuando usarlo	10
8.2. Código	10
8.3. El prompt a Cohere	11
9. El bug de DynamoDB: floats no soportados	11
9.1. El error real	11
9.2. Por qué pasó	12
9.3. El fix: helper _to_dynamo	12
10. Los 2 smoke tests E2E que validan todo	12
10.1. Test 1: Queja de ACOSO (palabra clave)	12
10.2. Test 2: Queja de INFRAESTRUCTURA	14
11. Catálogo de errores y soluciones	14
12. Verificación en AWS Console	15
12.1. Recursos desplegados (después de Fase 2)	15
12.2. Ver logs del análisis en tiempo real	15
12.3. Ver mensajes en DLQ (si hay errores)	16
13. Lo que aprendimos (resumen ejecutivo)	16
14. Ejercicios para practicar	16
14.1. Ejercicio 1: Añadir más keywords al MockLLM	16
14.2. Ejercicio 2: Implementar RealLLMClient en Lambda con Layer	16
14.3. Ejercicio 3: Añadir métricas CloudWatch	16
14.4. Ejercicio 4: Endpoint GET /api/quejas/{id}/analysis	16
14.5. Ejercicio 5: Añadir notificación por email (Fase 4 prep)	16
15. Siguiente paso: Fase 3	16

1. Arquitectura del flujo asíncrono

1.1. Diagrama del flujo



1.2. Status workflow (state machine)



Tiempos esperados:

- POST → 202: ~100ms (solo persiste + encola)
- PENDIENTE → ANALIZADA: ~2-3s (SQS dispatch + LLM call)

2. Componente 1: SQS Queue + Dead Letter Queue

2.1. Qué es SQS

Amazon SQS = cola de mensajes managed. Los producers envían, los consumers reciben.

Para qué sirve en nuestro caso:

- **Desacoplar** el POST rápido del análisis lento
- **Resiliencia**: si la Lambda consumer cae, los mensajes esperan
- **Retry automático**: si falla, SQS reintenta hasta 3 veces
- **DLQ** (Dead Letter Queue): mensajes que fallan 3 veces van acá para análisis manual

2.2. Configuración en SAM (template.yaml)

```

1 # Dead Letter Queue (mensajes que fallan 3 veces)
2 AnalysisDeadLetterQueue:
3   Type: AWS::SQS::Queue
4   Properties:
5     QueueName: !Sub "sentinel-analysis-dlq-${Stage}"
6     MessageRetentionPeriod: 1209600 # 14 dias
7
8 # Cola principal
9 AnalysisQueue:
10  Type: AWS::SQS::Queue
11  Properties:
  
```

```

12 QueueName: !Sub "sentinel-analysis-${Stage}"
13 VisibilityTimeout: 180 # 3 min para procesar
14 RedrivePolicy:
15     deadLetterTargetArn: !GetAtt AnalysisDeadLetterQueue.Arn
16     maxReceiveCount: 3 # 3 reintentos antes de DLQ

```

Listing 1: infra/template.yaml — SQS

Conceptos:

- **VisibilityTimeout:** tiempo que un mensaje está "invisible" después de ser leído. Si la Lambda no termina en 180s, el mensaje vuelve a la cola.
- **maxReceiveCount: 3:** después de 3 entregas fallidas, va al DLQ.
- **DLQ Retention: 14 días:** tiempo que se guarda el mensaje muerto para investigación.

2.3. Event mapping (SQS → Lambda)

```

1 AnalyzeQuejaFunction:
2   Type: AWS::Serverless::Function
3   Properties:
4     Role: !Ref LambdaRoleArn
5     FunctionName: !Sub "sentinel-analyze-queja-${Stage}"
6     CodeUri: ../src
7     Handler: handlers.analyze_queja.lambda_handler
8     MemorySize: 1024 # mas memoria para el LLM
9     Timeout: 120 # mas tiempo para Cohere
10    Environment:
11      Variables:
12        DYNAMODB_TABLE: !Sub "sentinel-quejas-${Stage}"
13        MOCK_LLM: "true" # usa MockLLMClient (no oci package)
14    Events:
15      SQSEvent:
16        Type: Sqs
17        Properties:
18          Queue: !GetAtt AnalysisQueue.Arn
19          BatchSize: 5 # procesa hasta 5 mensajes juntos
20    Policies:
21      - SQSPollerPolicy:
22        QueueName: !GetAtt AnalysisQueue.QueueName

```

Listing 2: infra/template.yaml — Lambda consumer

3. Componente 2: Analisis schema (Pydantic)**3.1. El schema alineado con OpenAPI**

```

1 from enum import StrEnum
2
3 class CriticidadEnum(StrEnum):
4     BAJA = "BAJA"
5     MEDIA = "MEDIA"
6     ALTA = "ALTA"
7     CRITICA = "CRITICA"
8
9 class SentimientoEnum(StrEnum):
10     POSITIVO = "POSITIVO"

```

```

11     NEUTRO = "NEUTRO"
12     NEGATIVO = "NEGATIVO"
13     NEGATIVO_FUERTE = "NEGATIVO_FUERTE"
14
15 class Analisis(BaseModel):
16     categoria: CategoriaEnum
17     subcategoria: str | None = None
18     criticidad: CriticidadEnum
19     criticidadJustificacion: str | None = None
20     sentimiento: SentimientoEnum
21     entidades: list[Entidad] = Field(default_factory=list)
22     temasClave: list[str] = Field(default_factory=list)
23     accionSugerida: str
24     prioridad: int = Field(ge=1, le=10)
25     requiereNotificacionInmediata: bool = False
26     modeloUsado: str = "cohere.command-latest"
27     tokensConsumidos: int = 0
28     latenciaMs: int = 0
29     confidence: float = Field(ge=0, le=1, default=0.0)
30     generatedAt: str

```

Listing 3: handlers/_shared/schemas.py — Analisis

3.2. El mensaje SQS

```

1 class SQSCreateQuejaMessage(BaseModel):
2     """Cuerpo del mensaje que create_queja envia a SQS."""
3     quejaId: str
4     tenantId: str
5     correlationId: str | None = None

```

Listing 4: handlers/_shared/schemas.py — SQS message

Por qué mínimo: la Lambda consumer hará un `get_queja()` a DynamoDB para obtener toda la info. El mensaje solo lleva el ID.

4. Componente 3: create_queja ahora envía a SQS

4.1. El nuevo flujo

```

1 import boto3
2 from handlers._shared.config import get_settings
3
4 def _send_to_sqs(queja_id: str, tenant_id: str, correlation_id: str) ->
    None:
5     """Envia mensaje a SQS para analisis async."""
6     settings = get_settings()
7     queue_url = settings.analysis_queue_url
8     if not queue_url:
9         log.warning("ANALYSIS_QUEUE_URL not configured, skipping SQS
            send")
10        return
11
12    sqs = boto3.client("sqs")
13    message_body = json.dumps({
14        "quejaId": queja_id,
15        "tenantId": tenant_id,

```

```

16         "correlationId": correlation_id,
17     })
18     sqs.send_message(
19         QueueUrl=queue_url,
20         MessageBody=message_body,
21         MessageAttributes={
22             "tenantId": {"StringValue": tenant_id, "DataType":
23                 "String"},
24     },
25 )

```

Listing 5: handlers/create_queja.py — agregar SQS

4.2. Integración en el handler

```

1  # 5. Persistir en DynamoDB
2  dynamo = get_dynamo_client()
3  dynamo.put_item(item=item,
4                  condition_expression="attribute_not_exists(pk)")
5
6  # 6. Enviar a SQS (analisis async)
7  try:
8      _send_to_sqs(accepted.quejaId, tenant_id, correlation_id)
9  except Exception as e:
10     # No fallar el request: la queja ya esta persistida
11     log.error("Failed to send SQS message (queja is still saved)",
12             extra={"error": str(e)})
13
14 # 7. Return 202 Accepted
15 return HttpResponse.accepted(accepted.model_dump(mode="json"))

```

Listing 6: lambda_handler — paso 6

Patrones importantes:

- ✓try/except en SQS send: si falla, la queja sigue guardada (mejor DX)
- ✓MessageAttributes: permiten filtrar en consumer si es necesario
- ✓MessageBody es JSON: estructurado, validable con Pydantic

5. Componente 4: analyze_queja Lambda (SQS consumer)

5.1. El handler completo

```

1  from handlers._shared.config import get_settings
2  from handlers._shared.correlation_id import (
3      clear_correlation_id, with_correlation_id
4  )
5  from handlers._shared.dynamo_client import (
6      get_dynamo_client, now_iso, queja_pk, queja_sk
7  )
8  from handlers._shared.errors import error_handler
9  from handlers._shared.logger import get_logger
10 from handlers._shared.schemas import QuejaResponse,
11     SQSCreateQuejaMessage
12 from services.llm_client import LLMError, get_llm_client

```

```

13 log = get_logger(__name__)
14
15
16 def _set_status(queja_id: str, tenant_id: str, status: str) -> None:
17     """Update queja status in DynamoDB."""
18     dynamo = get_dynamo_client()
19     dynamo.update_item(
20         pk=queja_pk(tenant_id, queja_id),
21         sk=queja_sk(queja_id),
22         updates={"status": status, "updatedAt": now_iso()},
23     )
24
25
26 def _process_one(record: dict) -> None:
27     """Process a single SQS record."""
28     body = record.get("body", "{}")
29     msg = SQSCreateQuejaMessage.model_validate_json(body)
30     log.append_keys(queja_id=msg.quejaId, tenant_id=msg.tenantId)
31
32     # 1. Fetch queja
33     dynamo = get_dynamo_client()
34     item = dynamo.get_queja(tenant_id=msg.tenantId,
35                             queja_id=msg.quejaId)
36     queja = QuejaResponse.from_dynamo(item)
37
38     # 2. Update to PROCESANDO
39     _set_status(msg.quejaId, msg.tenantId, "PROCESANDO")
40
41     # 3. Call LLM
42     llm = get_llm_client()
43     try:
44         analysis = llm.analyze_queja(queja)
45     except LLMEError as e:
46         _set_status(msg.quejaId, msg.tenantId, "ERROR")
47         raise # SQS retry
48
49     # 4. Update to ANALIZADA + store analysis
50     analysis_json = analysis.model_dump(mode="json")
51     dynamo.update_item(
52         pk=queja_pk(msg.tenantId, msg.quejaId),
53         sk=queja_sk(msg.quejaId),
54         updates={
55             "status": "ANALIZADA",
56             "updatedAt": now_iso(),
57             "analysis": analysis_json,
58         },
59     )
60
61 @error_handler
62 def lambda_handler(event, context):
63     """SQS event handler for the analysis queue."""
64     with_correlation_id(event)
65     try:
66         records = event.get("Records", [])
67         failed = []
68         for record in records:
69             try:

```

```

70         _process_one(record)
71     except Exception:
72         log.exception("Failed to process record")
73         failed.append({
74             "itemIdentifier": record.get("messageId"),
75             "message": "error"
76         })
77
78     # batchItemFailures: SQS reintentar\`a solo los fallidos
79     if failed:
80         return {"batchItemFailures": failed}
81
82     return {"statusCode": 200, "body": "OK"}
83 finally:
84     clear_correlation_id()

```

Listing 7: handlers/analyze_queja.py

5.2. Patrones importantes

- ✓ **batchItemFailures**: SQS solo reintenta los fallidos (no todos)
- ✓ **Status workflow**: PENDIENTE → PROCESANDO → ANALIZADA/ERROR
- ✓ **log.append_keys**: añade quejaId/tenantId a TODOS los logs
- ✓ **finally + clear**: limpia ContextVar al final

6. Componente 5: LLM Client (Mock + Real)

6.1. El problema del 250MB

Cuando agregamos oci a requirements.txt:

```

1 $ du -sh .aws-sam-new/AnalyzeQuejaFunction/
2 292M  .aws-sam-new/AnalyzeQuejaFunction/  <-- EXCEDE 250MB
3
4 $ sam deploy ...
5 Error: Unzipped size must be smaller than 262144000 bytes

```

Listing 8: Problema

El package oci pesa **229MB** (sus deps: cryptography, pycryptodome, etc.).

6.2. La solución: factory con toggle

```

1 import os
2
3 def get_llm_client():
4     """Get LLM client based on MOCK_LLM env var."""
5     global _client
6     if _client is not None:
7         return _client
8
9     use_mock = os.environ.get("MOCK_LLM", "true").lower() != "false"
10    if use_mock:
11        log.info("Using MockLLMClient (rule-based)")
12        _client = MockLLMClient()

```



```

13     else:
14         log.info("Using RealLLMClient (OCI Cohere)")
15         _client = RealLLMClient()
16     return _client

```

Listing 9: services/llm_client.py — factory

En SAM template:

```

1 Environment:
2 Variables:
3     MOCK_LLM: "true"      # Lambda = mock

```

En tu máquina local:

```

1 unset MOCK_LLM          # o MOCK_LLM=false
2 python mi_script.py     # usa RealLLMClient

```

7. MockLLMClient: rule-based

7.1. Algoritmo de clasificación

```

1 class MockLLMClient:
2     KEYWORD_MAP = {
3         "ACOSO": (["acoso", "ofensiv", "agresi", "violencia",
4                   "amenaza", "hostig"], "CRITICA", 10),
5         "SALUD": (["suicid", "depres", "ansiedad", "salud mental"],
6                  "CRITICA", 9),
7         "INFRAESTRUCTURA": (["aula", "edificio", "baño", "techo",
8                              "inund", "filtr", "electric", "dañado"],
9                             "MEDIA", 5),
10        "ACADEMICA": (["profesor", "docente", "calificaci",
11                      "examen", "clase"], "MEDIA", 6),
12        "ADMINISTRATIVA": (["matrícula", "inscripci", "trámite",
13                            "sistema", "pago", "becas"], "MEDIA", 5),
14    }
15
16    def analyze_queja(self, queja: QuejaResponse) -> Analisis:
17        text = f"{queja.titulo} {queja.descripcion}".lower()
18
19        # 1. Detectar categoria por keywords
20        best_match = ("OTRA", [], "BAJA", 3)
21        for cat, (keywords, crit, prio) in self.KEYWORD_MAP.items():
22            score = sum(1 for kw in keywords if kw in text)
23            if score > 0 and (best_match[0] == "OTRA" or score >
24                             best_match[3]):
25                best_match = (cat, keywords, crit, prio)
26
27        # 2. Detectar sentimiento
28        if any(w in text for w in ["terrible", "horrible", "urgente",
29                                   "peligro"]):
30            sentimiento = "NEGATIVO_FUERTE"
31        elif any(w in text for w in ["mal", "problema", "falla", "error"]):
32            sentimiento = "NEGATIVO"
33        # ... etc
34
35        # 3. Build Analisis

```

```

34     return Analisis(
35         categoria=CategoriaEnum(categoria_str),
36         criticidad=CriticidadEnum(criticidad_str),
37         sentimiento=SentimientoEnum(sentimiento),
38         prioridad=prioridad,
39         accionSugerida=accion,
40         # ...
41     )

```

Listing 10: services/llm_client.py — MockLLMClient

Trade-offs:

- **✓Ventaja:** 0 deps externas, 0 latencia, deterministic
- **✗Desventaja:** solo detecta keywords obvios, no entiende contexto
- **✓Para el demo:** suficiente; el flujo async se demuestra igual

8. RealLLMClient: OCI Cohere (solo local)

8.1. Cuando usarlo

- Para hacer demos en tu máquina con el Cohere real
- Para tests que requieren LLM real
- NO en Lambda (excede 250MB)

8.2. Código

```

1 class RealLLMClient:
2     def __init__(self):
3         import oci
4         from oci.generative_ai_inference import
5             GenerativeAiInferenceClient
6         from oci.generative_ai_inference import models as
7             inference_models
8
9         settings = get_settings()
10        self.config = oci.config.from_file(settings.oci_key_file,
11            "DEFAULT")
12        # override con env vars si existen
13        if settings.oci_user_ocid:
14            self.config["user"] = settings.oci_user_ocid
15        # ...
16
17        self._client = GenerativeAiInferenceClient(
18            self.config,
19            service_endpoint=f"https://inference.generativeai.{self.region}.oci.oraclecloud.com"
20        )
21        self._inference_models = inference_models
22
23    def analyze_queja(self, queja):
24        user_prompt = self._build_user_prompt(queja)
25        chat_request = self._inference_models.CohereChatRequest(
26            message=user_prompt,
27            max_tokens=800,
28            temperature=0.2,

```

```

26         is_stream=False,
27     )
28     details = self._inference_models.ChatDetails(
29         compartment_id=self.compartment_id,
30         serving_mode=self._inference_models.OnDemandServingMode(
31             model_id=self.model_ocid,
32         ),
33         chat_request=chat_request,
34     )
35     response = self._client.chat(details)
36     text = response.data.chat_response.text
37     return self._parse_to_analisis(text)

```

Listing 11: services/llm_client.py — RealLLMClient

8.3. El prompt a Cohere

```

1  SYSTEM_PROMPT = """Eres un asistente especializado en analizar quejas
   universitarias.
2  Devuelve SOLO JSON v\u00e9lido con esta estructura:
3
4  {
5      "categoria": "ACADEMICA" | "INFRAESTRUCTURA" | "ACOSO" |
6                  "ADMINISTRATIVA" | "SALUD" | "OTRA",
7      "criticidad": "BAJA" | "MEDIA" | "ALTA" | "CRITICA",
8      "sentimiento": "POSITIVO" | "NEUTRO" | "NEGATIVO" | "NEGATIVO_FUERTE",
9      "prioridad": 1-10,
10     "requiereNotificacionInmediata": true | false,
11     "confidence": 0.0-1.0,
12     // ... etc
13 }
14
15 NO devuelvas markdown. SOLO el JSON puro."""

```

Listing 12: System prompt

9. El bug de DynamoDB: floats no soportados

9.1. El error real

Cuando hicimos el primer test E2E, los logs mostraron:

```

1  {
2      "errorType": "TypeError",
3      "errorMessage": "Float types are not supported. Use Decimal types
   instead.",
4      "stackTrace": [
5          "File \"/var/task/boto3/dynamodb/types.py\", line 171, in
   _is_number",
6          "raise TypeError("
7      ]
8  }

```

Listing 13: CloudWatch logs — error real

9.2. Por qué pasó

El `Analysis.confidence` es un `float` (Python). DynamoDB **no acepta floats nativos**, solo `Decimal`:

```

1 # MAL
2 analysis.confidence = 0.75 # -> Python float
3 dynamo.update_item({"analysis": analysis.dict()}) # -> ERROR
4
5 # BIEN
6 from decimal import Decimal
7 analysis.confidence = Decimal("0.75") # -> DynamoDB N

```

9.3. El fix: helper `_to_dynamo`

```

1 from decimal import Decimal
2
3 def _to_dynamo(value):
4     """Recursively convert floats to Decimal for DynamoDB."""
5     if isinstance(value, float):
6         return Decimal(str(value)) # str() evita precision issues
7     if isinstance(value, dict):
8         return {k: _to_dynamo(v) for k, v in value.items()}
9     if isinstance(value, list):
10        return [_to_dynamo(item) for item in value]
11    return value
12
13
14 def put_item(self, item, condition_expression=None):
15     kwargs = {"Item": _to_dynamo(item)} # <-- APLICAR
16     # ...
17
18
19 def update_item(self, pk, sk, updates):
20     # ...
21     response = self._table.update_item(
22         Key={"pk": pk, "sk": sk},
23         UpdateExpression=update_expr,
24         ExpressionAttributeValues=_to_dynamo(expression_values), # <--
25         # ...
26     )

```

Listing 14: `handlers/_shared/dynamo_client.py`

Patrones:

- ✓ `Decimal(str(x))` en vez de `Decimal(x)` para evitar `0.1 + 0.2 = 0.30000000000000004`
- ✓ Recursivo: maneja dicts y lists anidados
- ✗ No lo olvides: si añades otro `client`, también debe usar `_to_dynamo`

10. Los 2 smoke tests E2E que validan todo

10.1. Test 1: Queja de ACOSO (palabra clave)

```

1 API="https://om3eiczjr9.execute-api.us-east-1.amazonaws.com/dev"

```

```

2
3 # Paso 1: POST
4 RESP=$(curl -s -X POST "$API/api/quejas" \
5   -H "Content-Type: application/json" \
6   -H "X-Tenant-ID: utpl" \
7   -H "X-User-ID: estudiante-003" \
8   -H "X-Correlation-ID: fase2-final-001" \
9   -d '{"titulo":"Situacion de acoso en el salon",
10      "descripcion":"Varios companeros han reportado comentarios
11      agresivos y ofensivos reiterados. Situacion
12      urgente.",
13      "categoriaDeclarada":"ACOSO",
14      "anonima":false}')
15
16 # Output esperado (202 Accepted)
17 # {"quejaId":"e3148f6c-...", "status":"PENDIENTE",
18 #  "correlationId":"fase2-final-001",
19 #  "createdAt":"2026-06-20T14:02:45.077465+00:00",
20 #  "estimatedAnalysisTime":30}
21
22 # Paso 2: Esperar 5 segundos (SQS dispatch)
23 sleep 5
24
25 # Paso 3: GET con analysis
26 curl -s "$API/api/quejas/$QUEJA_ID" -H "X-Tenant-ID: utpl"

```

Listing 15: Test E2E: ACOSO

```

1 {
2   "quejaId": "e3148f6c-92ec-4df2-9669-3539aeeb2195",
3   "status": "ANALIZADA",
4   "analysis": {
5     "categoria": "ACOSO",
6     "subcategoria": "acoso",
7     "criticidad": "CRITICA",
8     "criticidadJustificacion":
9       "Detectadas palabras clave de ACOSO. Prioridad 10/10.",
10    "sentimiento": "NEGATIVO_FUERTE",
11    "entidades": [
12      {"tipo": "MENCION", "texto": "Varios"},
13      {"tipo": "MENCION", "texto": "Situacion"}
14    ],
15    "temasClave": ["acoso", "agresivos", "ofensivos"],
16    "accionSugerida":
17      "Notificar a BIENESTAR y SEGURIDAD inmediatamente.
18      Escalar a direccion.",
19    "prioridad": 10,
20    "requiereNotificacionInmediata": true,
21    "modeloUsado": "mock-rule-based-v1",
22    "tokensConsumidos": 13,
23    "latenciaMs": 0,
24    "confidence": 0.75,
25    "generatedAt": "2026-06-20T14:02:48.156952+00:00"
26  }
27 }

```

Listing 16: Output del GET — analysis detectada

10.2. Test 2: Queja de INFRAESTRUCTURA

```

1 RESP=$(curl -s -X POST "$API/api/quejas" \
2   -H "Content-Type: application/json" \
3   -H "X-Tenant-ID: utpl" \
4   -H "X-Correlation-ID: fase2-infra-001" \
5   -d '{"titulo":"Techo del salon con filtraciones",
6       "descripcion":"El aula 205 tiene el techo dañado y se
7           inunda cuando llueve. Hay charcos constantes.",
8       "categoriaDeclarada":"INFRAESTRUCTURA",
9       "anonima":false}')
10
11 sleep 5
12 # Output del GET:
13 # {
14 #   "categoria": "INFRAESTRUCTURA",
15 #   "criticidad": "MEDIA",
16 #   "prioridad": 5,
17 #   "requiereNotificacionInmediata": false,
18 #   "temasClave": ["aula"],
19 #   "accionSugerida": "Asignar a comite academico. Responder
20 #       en 5 dias habiles.",
21 #   "modeloUsado": "mock-rule-based-v1",
22 #   "confidence": 0.75
23 # }
```

Listing 17: Test E2E: INFRAESTRUCTURA

11. Catálogo de errores y soluciones

Error 1: Unzipped size >250MB

Cuando: sam deploy con oci en requirements.

Causa: el package oci pesa 229MB solo.

Solución: usar MockLLMClient en Lambda; oci solo local.

Error 2: Float types not supported

Cuando: guardar Analisis con confidence: 0.75.

Causa: DynamoDB no acepta float, solo Decimal.

Solución: helper _to_dynamo que convierte recursivamente.

Error 3: SQS message sent but consumer no triggered

Cuando: el mensaje va a SQS pero la Lambda no se ejecuta.

Causa: permisos del LabRole (no sqs:ReceiveMessage).

Solución: agregar SQSPollerPolicy en el template.

Error 4: batchItemFailures no reportado

Cuando: SQS reintenta TODOS los mensajes, no solo los fallidos.

Causa: la Lambda retorna 200 en vez de {batchItemFailures: [...]}.
Solución: capturar excepciones per-record y reportar failures.

Error 5: analysis en null después de esperar**Cuando:** GET devuelve `analysis: null` tras 30s.**Causa:** la Lambda falló silenciosamente; revisar CloudWatch logs.**Solución:** `aws logs tail /aws/lambda/sentinel-analyze-queja-dev`.

12. Verificación en AWS Console

12.1. Recursos desplegados (después de Fase 2)

```

1 aws cloudformation describe-stack-resources \
2   --stack-name sentinel-academia-dev \
3   --query 'StackResources[].[Type:ResourceType, Id:PhysicalResourceId,
4     Name:LogicalResourceId]' \
5   --output table

```

Listing 18: Listar todos los recursos del stack

Type	Name	Status
AWS::DynamoDB::Table	QuejasTable	CREATE_COMPLETE
AWS::SQS::Queue	AnalysisQueue	CREATE_COMPLETE
AWS::SQS::Queue	AnalysisDeadLetterQueue	CREATE_COMPLETE
AWS::S3::Bucket	FilesBucket	CREATE_COMPLETE
AWS::Lambda::Function	HealthCheckFunction	CREATE_COMPLETE
AWS::Lambda::Function	CreateQuejaFunction	CREATE_COMPLETE
AWS::Lambda::Function	GetQuejaFunction	CREATE_COMPLETE
AWS::Lambda::Function	ListQuejasFunction	CREATE_COMPLETE
AWS::Lambda::Function	AnalyzeQuejaFunction	CREATE_COMPLETE
AWS::ApiGateway::RestApi	SentinelApi	CREATE_COMPLETE

Listing 19: Output esperado (resumen)

12.2. Ver logs del análisis en tiempo real

```

1 aws logs tail /aws/lambda/sentinel-analyze-queja-dev --follow

```

Listing 20: Tail logs de analyze_queja

Output esperado:

```

1 {"level":"INFO","location":"lambda_handler:103",
2   "message":"Processing 1 SQS records",
3   "queja_id":"e3148f6c-92ec-4df2-9669-3539aeeb2195",
4   "tenant_id":"utpl"}
5
6 {"level":"INFO","location":"_process_one:63",
7   "message":"Status updated to PROCESANDO"}
8
9 {"level":"INFO","location":"get_llm_client:295",
10  "message":"Using MockLLMClient (rule-based)"}
11
12 {"level":"INFO","location":"_process_one:84",
13  "message":"Analysis stored",
14  "categoria":"ACOSO","criticidad":"CRITICA","prioridad":10}

```

12.3. Ver mensajes en DLQ (si hay errores)

```

1 aws sqs get-queue-attributes \
2   --queue-url
3   https://sqs.us-east-1.amazonaws.com/227165337884/sentinel-analysis-dlq-dev
   \
   --attribute-names All

```

Listing 21: Inspeccionar DLQ

13. Lo que aprendimos (resumen ejecutivo)

Checklist Fase 2

1. **SQS para desacoplar:** POST rápido, análisis lento
2. **DLQ para resiliencia:** mensajes fallidos van a cola aparte
3. **VisibilityTimeout:** 3 min para procesar; si no, reintenta
4. **batchItemFailures:** SQS solo reintenta los fallidos
5. **Status workflow:** PENDIENTE → PROCESANDO → ANALIZADA
6. **Mock vs Real LLM:** Toggle con MOCK_LLM env var
7. **Lambda 250MB limit:** oci package NO cabe; usar mock
8. **DynamoDB Decimal:** floats de Python NO funcionan; helper `_to_dynamo`
9. **log.append_keys:** añade context (quejaId, tenantId) a todos los logs
10. **System prompt LLM:** instrucciones claras + JSON schema

14. Ejercicios para practicar

14.1. Ejercicio 1: Añadir más keywords al MockLLM

Edita `MockLLMClient.KEYWORD_MAP` para reconocer 5 categorías más. Testéalo con quejas reales.

14.2. Ejercicio 2: Implementar RealLLMClient en Lambda con Layer

Crea un Lambda Layer con el package `oci slim` (solo lo necesario: `oci.generative_ai_inference`). Adjúntalo a `AnalyzeQuejaFunction`. Cambia `MOCK_LLM=false`.

14.3. Ejercicio 3: Añadir métricas CloudWatch

Mide cuánto tarda el LLM (latency), cuántas quejas se procesan por minuto, cuántas fallan. Usa `aws-lambda-powertools.metrics`.

14.4. Ejercicio 4: Endpoint GET /api/quejas/{id}/analysis

Crea un nuevo handler que devuelva SOLO el analysis (no toda la queja). Útil para el dashboard.

14.5. Ejercicio 5: Añadir notificación por email (Fase 4 prep)

Cuando el status pasa a NOTIFICADA, envía email vía SES al admin. Usa el campo `requiereNotificacionInmediata` para filtrar.

15. Siguiendo paso: Fase 3

En la **Fase 3** desplegamos el **frontend Vue 3**:

- Configurar variable de entorno con la API URL
- Build de producción con Vite
- Deploy a **Vercel** (o S3 + CloudFront)
- Probar el flujo end-to-end: frontend → backend → SQS → LLM
- Capturar screenshots para el demo

Tiempo estimado: 1-2 horas.

Fase	Estado
0. Setup + OCI	✓
1. Backend Core	✓
2. LLM integration (este tutorial)	✓
3. Frontend deploy	próximo
4. Archivos + Email	pendiente
5. RAG	opcional
6. Testing	pendiente
7. Deploy final + demo	pendiente

Fin del tutorial. ¡A por la Fase 3!